MimeSys Algorithms

Package version 0.99.0

17 October 2026

Contents

1. Notation **3**

2. Fitting the Template **4**
2.1 Input . . . 4
2.2 Sample matching . . . 4
2.3 Optional view transformation . . . 4
2.4 Missing values . . . 5
2.5 Constant features . . . 5
2.6 Empirical quantile functions . . . 5
2.7 Normal-score transformation . . . 5
2.8 Within-view factor model . . . 6
2.9 How within-view rank is chosen . . . 6
2.10 Loading stabilization . . . 6
2.11 Shared cross-view latent space . . . 6
2.12 How shared rank is chosen . . . 7
2.13 Shared factor scores . . . 8
2.14 View-specific shared loading matrices . . . 8

3. Null Multi-Omics Feature Generation **8**
3.1 Goal . . . 8
3.2 Feature count handling . . . 8
3.3 If requested features are fewer than template features . . . 9
3.4 If requested features exceed template features . . . 9
3.5 Crosstalk . . . 9
3.6 Crosstalk interpretation . . . 10
3.7 Valid crosstalk range . . . 10
3.8 Cross-view covariance induced by crosstalk . . . 10
3.9 Simulate shared latent factors . . . 10
3.10 Simulate private view-specific factors . . . 11
3.11 Generate latent normal-score features . . . 11
3.12 Map latent features to empirical scale . . . 11

4. Microbiome Two-Stage Emission **11**
4.1 Goal . . . 11

4.2 When used . . . 11
4.3 Stage 1: presence/absence . . . 12
4.4 Stage 2: positive abundance . . . 12
4.5 Final microbiome value . . . 12
4.6 Relationship to crosstalk . . . 13
4.7 Relationship to outcome generation . . . 13

5. Spike-In Coefficients **13**
5.1 Goal . . . 13
5.2 Number of nonzero effects . . . 13
5.3 Effect magnitudes . . . 14
5.4 Differential-effect alignment through `de.align` . . . 14
5.5 Interpretation of `de.align` . . . 14

6. View-Level Signal Balancing **15**
6.1 Goal . . . 15
6.2 View predictors on standardized features . . . 15
6.3 Normalizing requested shares . . . 15
6.4 Rescaling beta values . . . 15
6.5 Final additive linear predictor . . . 15

7. Outcome Generation Modes **15**

7.1 `ygen.mode = "LM"` **15**

7.2 `ygen.mode = "Friedman"` **16**

7.3 `ygen.mode = "Friedman2"` **16**

7.4 `ygen.mode = "SharedLatent"` **16**
Are all shared factors used? . . . 17
Why multiple shared factors matter . . . 17
Shared latent linear outcome . . . 17
Shared latent Friedman outcome . . . 18
Shared latent Friedman2 outcome . . . 18
Role of crosstalk in shared-latent outcomes . . . 18

7.5 `ygen.mode = "Interaction"` **19**

7.6 Ground truth **19**

8. Final Outcome Types **19**
8.1 Continuous outcome . . . 19
8.2 Binary outcome . . . 20
8.3 Survival outcome . . . 20

9. Train/Test Split **20**

10. Crosstalk Calibration **21**
10.1 Template validation . . . 21

10.2 Reproducibility 22

11. Summary of Controls **22**

12. Mediation Simulation **22**
12.1 Exposure 23
12.2 Exposure to mediators 23
12.3 Outcome 23
12.4 Path sizes and calibration 23
12.5 Truth 23
12.6 Sequential mediation 24

13. Recommended Controlled Shared-Latent Setup **25**

This document describes the simulation algorithms used by MimeSys in detail. The goal is to make the data-generation mechanism transparent enough that each simulation setting can be traced back to a specific statistical operation.

The key separation is:

- **Null feature generation** creates simulated multi-omics matrices and their covariance/correlation structure.
- **Outcome generation** creates the phenotype/response after the simulated omics matrices exist.
- **Microbiome two-stage emission** changes how sparse microbiome-like features are emitted, but it is still part of null feature generation.

The most important conceptual point is:

```
crosstalk controls cross-view feature dependence.
outcome generation controls what predicts Y.
```

High crosstalk means the omics layers share more latent structure. It does not automatically mean that fusion must outperform single-layer or simpler models. Fusion performance depends on how the outcome is generated and whether signal is complementary, shared, redundant, nonlinear, or concentrated in one layer.

1. Notation

Assume `M` omics views. For view `m`:

```
X_m is an n x p_m matrix.
```

Where:

- `n` is the simulated sample size.
- `p_m` is the number of simulated features in view `m`.
- `X_m[i, j]` is feature `j` in view `m` for sample `i`.

The complete simulated feature matrix is:

```
X = [X_1, X_2, ..., X_M].
```

MimeSys fits a reusable template from empirical data. The template stores:

- empirical feature marginals,
- normal-score transformed features,
- within-view factor loadings,
- within-view factor scores,
- shared cross-view latent factors,
- mappings from shared latent factors into each view,
- optional two-stage emission parameters for sparse microbiome-like views.

2. Fitting the Template

2.1 Input

The input is a `MultiAssayExperiment`, a single `SummarizedExperiment`, or a named list of matched omics matrices:

```
template <- list(
  CYTOF = cytof_matrix,
  Metabolomics = metabolomics_matrix,
  Proteomics = proteomics_matrix
)
```

Internally every view is a numeric matrix with samples in rows and features in columns; MAE and SummarizedExperiment assays (features in rows) are transposed, including MAE experiments stored as plain matrices. Missing row names become `sample_1, ...`, missing feature names `<view>_feature_1, ...`, and duplicated feature names are made unique. `validate_mimesys_input()` reports samples, features, missing values and constant features per view before fitting.

2.2 Sample matching

For an MAE, each experiment's column names are first replaced by the primary sample IDs of its `sampleMap()`, so experiments that name the same subject differently are still matched (several columns for one primary sample are an error). MimeSys then intersects the sample IDs across views and keeps only the samples shared by all views; at least 10 are required, and duplicate IDs within a view are an error.

Because missing row names are filled with `sample_1, sample_2, ...`, a list of matrices without row names is matched by position. If such views have different numbers of rows, only the first `min(n_m)` rows are kept, without a warning, so give matrices explicit sample IDs.

The template assumes one sample per subject. For repeated measures, fit the template on one timepoint: all 150 samples of the full Stelzer data (3 timepoints for 53 subjects) give 18 shared factors, one timepoint gives 2.

2.3 Optional view transformation

Before fitting, views can be transformed using `view.transform`.

Supported choices include:

- `"none"`: no transformation,
- `"log1p"`: apply `log(1 + x)`,

- `"clr"`: centered log-ratio style transformation for compositional data.

`log1p` adds a shift `|min(x)| + 1` first when the view has values below -1; `clr` requires non-negative data and uses half the smallest positive value as pseudocount. Unknown view names are errors; unnamed views use `"none"`.

The template is fitted, calibrated and validated on the transformed ("fitted") scale, and every simulator returns that scale by default. With `back.transform = TRUE` (`null_model$back_transform` in the MAE wrappers), each simulated feature's uniform is mapped through the feature's original-scale empirical quantile function instead: ranks are kept exactly and each feature recovers its original marginal (for `clr`, including its zeros). For `log1p` this agrees with `expm1()` of the fitted value at observed values but interpolates on the original scale between them. Two-part views fitted with `log1p` use the exact inverse `exp(x) - 1 - shift`.

2.4 Missing values

Features missing in every sample are dropped (with a message), features more than 50% missing give a warning, and the remaining missing values are median-imputed within each feature:

```
median(x_j, na.rm = TRUE).
```

2.5 Constant features

Features with no variation are removed. A constant feature cannot contribute to correlation, factor structure, or outcome signal.

2.6 Empirical quantile functions

For each feature, MimeSys stores the sorted empirical values:

```
Q_j.
```

This stored empirical quantile function is later used to map simulated normal-score values back to the empirical scale.

For integer-like features, MimeSys records that the feature is discrete. Discrete features use step-function quantile mapping so that simulated values remain integer-like.

2.7 Normal-score transformation

Each empirical feature is converted to normal scores:

```
z_ij = Phi^{-1}(rank(x_ij) / (n0 + 1)).
```

Where:

- `n0` is the template sample size,
- `Phi^{-1}` is the standard normal quantile function,
- randomized tie handling is used to deal with zeros and repeated values.

The normal-score matrix for view `m` is:

```
Z_m.
```

2.8 Within-view factor model

For each view, MimeSys fits a low-rank factor model on `Z_m`:

```
Z_m approx F_m Lambda_m^T + residual noise.
```

Where:

- `F_m` is the sample-by-factor score matrix for view `m`,
- `Lambda_m` is the feature-by-factor loading matrix,
- `k_m` is the number of within-view factors.

This is obtained through truncated SVD.

2.9 How within-view rank is chosen

MimeSys does **not** use `2 log(p)` by default.

If the user supplies:

```
rank = K
```

then `K` factors are used per view, or the supplied vector is recycled/matched across views.

If:

```
rank = NULL
```

MimeSys chooses the smallest rank explaining at least `var.explained` of the normal-score variance:

```
k_m = smallest k such that cumulative variance explained >= var.explained.
```

Then this is capped by:

```
max.rank = floor(n0 / 3)
```

unless the user supplies a different `max.rank`.

Therefore:

```
k_m = min(rank needed for var.explained, max.rank, p_m - 1).
```

2.10 Loading stabilization

For each feature:

```
h_j = ||Lambda_m[j, ]||^2.
```

If:

```
h_j > 0.99,
```

MimeSys shrinks that loading row so the residual variance remains positive.

This prevents invalid residual variance when simulating new normal-score data.

2.11 Shared cross-view latent space

After fitting each view, MimeSys stacks the within-view factor scores:

```
F_all = [F_1, F_2, ..., F_M].
```

It computes:

```
cor(F_all)
```

and performs an eigen decomposition. The leading eigenvectors define the shared cross-view latent structure, similar to a MAXVAR/generalized CCA representation.

2.12 How shared rank is chosen

MimeSys does **not** use `2 log(p)` for the shared rank either.

If the user supplies:

```
shared.rank = r
```

then MimeSys uses exactly `r` shared factors.

If:

```
shared.rank = NULL
```

MimeSys estimates the number of shared factors by sequential (deflated) permutation-based parallel analysis. Starting from the view score matrices `F_1, ..., F_M` and `k = 0`:

1. Compute the largest eigenvalue `lambda_obs` of `cor([F_1, ..., F_M])`.
2. Independently permute sample labels within each view's current score matrix and recompute the largest eigenvalue; repeat `nperm` times.
3. The step p-value is `(1 + #{lambda_null >= lambda_obs}) / (nperm + 1)`.
4. If `p >= 0.05`, stop: `shared.rank = k`.
5. Otherwise set `k = k + 1`, take the leading shared variate `s = [F_1, ..., F_M] v_1` (standardized), regress `s` out of every view's scores, re-orthonormalize each view's residual scores (dropping the one dimension removed), and go back to step 1.

The procedure also stops when `k` reaches the smallest within-view rank. If the first step is not significant, then:

```
shared.rank = 0.
```

Testing each step against the null of the *largest* eigenvalue, after removing the directions already accepted, is what makes the test valid. The classical variant that compares the `k`-th observed eigenvalue with the null of the `k`-th permuted eigenvalue over-selects here: a real shared factor lifts every later observed eigenvalue above its permuted counterpart. In simulations with 2, 3 and 5 true shared factors, the index-matched test chose 6-15, while the sequential test recovered the truth in most runs (at small `n` it occasionally picks one factor too many, because the deflated direction is estimated).

The shared loadings `B_m` are taken from the top `shared.rank` eigenvectors of the original `cor(F_all)`. The step p-values, the 95th percentile of each step's null (`gcca.threshold`) and the tested eigenvalues (`gcca.step.eigen`) are stored in the template, and a warning flags a borderline choice (last accepted step with $p > 0.025$, or first rejected step with $p < 0.08$).

Stability selection (optional). The normal scores use random tie-breaking and the null uses random permutations, so a borderline rank can vary between runs. With `shared.rank.runs = R >`

1 (default 1), the whole selection (normal scores, factor models and parallel analysis) is repeated R times and

```
shared.rank = round(mean(r_1, ..., r_R)),
```

capped at the smallest within-view rank. The runs, their distribution and the share of runs equal to the chosen rank are stored in shared.rank.selection; a message is shown when fewer than half agree.

2.13 Shared factor scores

If shared.rank > 0, MimeSys constructs:

```
S in R^(n0 x r).
```

This is obtained by projecting the stacked factor scores onto the leading shared eigenvectors, then orthonormalizing and scaling the scores.

2.14 View-specific shared loading matrices

For each view:

```
B_m = F_m^T S / (n0 - 1).
```

B_m maps the shared latent factors into view m's within-view factor space.

The template therefore contains:

```
Lambda_m  : feature loadings within view m
F_m       : within-view factor scores
S         : shared latent scores
B_m       : map from shared factors to view m
Q_j       : empirical feature quantile functions
```

3. Null Multi-Omics Feature Generation

3.1 Goal

Generate synthetic omics matrices that preserve:

- empirical marginal distributions,
- within-view dependence,
- cross-view dependence,
- optional sparse/zero-inflated microbiome behavior.

This is where crosstalk operates.

3.2 Feature count handling

For each view, requested feature count p_m can be:

- NULL: keep the template feature count,
- a scalar: same feature count for every view,
- a vector: view-specific feature counts.

3.3 If requested features are fewer than template features

If:

```
p_m <= p0_m,
```

MimeSys selects parent features from the template.

Selection can be:

- `"random"`: random subset,
- `"variance"`: highest-variance features first (on the fitted scale, after any `view.transform`).

The selected parent feature supplies:

- empirical marginal distribution,
- loading vector,
- two-stage emission parameters if relevant.

3.4 If requested features exceed template features

If:

```
p_m > p0_m,
```

MimeSys creates offspring features.

Each offspring feature copies a parent marginal distribution but receives perturbed loadings:

```
Lambda_offspring =
  sqrt(1 - feature.jitter) Lambda_parent
  + sqrt(feature.jitter) Lambda_other.
```

Interpretation:

- small `feature.jitter`: offspring closely resembles the parent,
- large `feature.jitter`: offspring is more perturbed.

3.5 Crosstalk

For view `m`, crosstalk is:

```
gamma_m.
```

It controls how strongly the simulated within-view factor scores are driven by the shared latent factors.

In the factor simulator:

```
f_m = sqrt(gamma_m) S_new B_m^T
      + E_m sqrt(I - gamma_m B_m B_m^T).
```

Where:

- `f_m` is the simulated factor score matrix for view `m`,
- `S_new` is the simulated shared latent matrix,
- `B_m` is the template-estimated map from shared factors to view `m`,

- `E_m` is independent view-specific Gaussian noise,
- the square-root residual term preserves valid covariance.

3.6 Crosstalk interpretation

If:

```
gamma_m = 0,
```

then:

```
f_m = E_m.
```

The view has no shared latent component.

If:

```
gamma_m = 1,
```

the view uses the fitted template-level shared structure.

If:

```
0 < gamma_m < 1,
```

the view partially uses the shared structure.

If valid values above 1 are allowed, they amplify the shared structure beyond the fitted template level.

3.7 Valid crosstalk range

MimeSys requires:

```
I - gamma_m B_m B_m^T
```

to remain positive semidefinite.

Therefore:

```
gamma_m <= 1 / sigma_max(B_m)^2.
```

This is what `crosstalk_max(template)` computes.

3.8 Cross-view covariance induced by crosstalk

For two views `m` and `m'`:

```
Cov(f_m, f_m') = sqrt(gamma_m gamma_m') B_m B_m'^T.
```

So crosstalk directly scales cross-view latent covariance.

This is why increasing crosstalk generally increases cross-view feature correlation.

3.9 Simulate shared latent factors

For the factor simulator:

```
S_new ~ N(0, I_r).
```

Rows are samples and columns are shared latent factors.

If `r = 0`, there are no shared factors.

3.10 Simulate private view-specific factors

For each view:

```
E_m ~ N(0, I_{k_m}).
```

Then:

```
f_m = shared component + private component.
```

3.11 Generate latent normal-score features

For view `m`:

```
Z_m = f_m Lambda_m^T + epsilon_m.
```

For feature `j`:

```
epsilon_ij ~ N(0, psi_j),
psi_j = 1 - ||Lambda_m[j, ]||^2.
```

3.12 Map latent features to empirical scale

The latent normal-score values are converted to uniforms:

```
U_mj = Phi(Z_mj).
```

Then they are mapped through the empirical quantile function:

```
X_mj = Q_mj(U_mj).
```

This preserves feature-specific skewness, discreteness, bounds, and empirical marginal behavior.

4. Microbiome Two-Stage Emission

4.1 Goal

Generate sparse, zero-inflated microbiome-like data while preserving dependence through the latent simulator.

This is part of null feature generation. It is not an outcome model.

4.2 When used

For example:

```
view.models = c(Microbiome = "two_part_copula")
```

Then the microbiome view uses a two-stage emission model.

4.3 Stage 1: presence/absence

For feature `j`, MimeSys generates a latent presence variable:

```
Z_presence_ij.
```

Then:

```
U_presence_ij = Phi(Z_presence_ij).
```

The empirical zero fraction from the template determines a threshold. Conceptually:

```
A_ij = 1 if U_presence_ij exceeds the zero threshold,
       0 otherwise.
```

If:

```
A_ij = 0,
```

the emitted value is zero.

4.4 Stage 2: positive abundance

For present features:

```
A_ij = 1,
```

MimeSys generates a second latent abundance variable, a lightly perturbed copy of the presence variable:

```
Z_abundance_ij = (Z_presence_ij + N(0, 0.15^2)) / sqrt(1 + 0.15^2),
U_abundance_ij = Phi(Z_abundance_ij).
```

The rescaling keeps `Z_abundance_ij` standard normal, so `U_abundance_ij` is exactly uniform. The two-part model is for sparse non-negative abundances only: a view with negative values, or fitted on the CLR scale (which has no zeros), is an error.

Because a present feature already has `U_presence_ij > pi_j`, the abundance uniform is rescaled to the part of `(0, 1)` above the zero probability before it is mapped through the positive-value quantile function:

```
U*_ij = (U_abundance_ij - pi_j) / (1 - pi_j),
V_ij  = Q_positive_j(U*_ij).
```

Without this conditioning, present features would draw only from the upper quantiles of their positive distribution and abundances would be inflated.

4.5 Final microbiome value

The observed value is:

```
X_ij = A_ij V_ij.
```

So:

- if `A_ij = 0`, then `X_ij = 0`,
- if `A_ij = 1`, then `X_ij` is positive and drawn from the empirical positive-value distribution.

Composition views never exceed the template total. If every template sample sums to the same total (a closed composition, such as relative abundances summing to 1) and at least the template's features are simulated, including offspring features, every simulated sample is renormalized to that total, so offspring features dilute the others. If fewer features are simulated, samples above the total are scaled down to it. If the template is itself a subset of a composition (values in [0, 1] with sample totals at most 1), samples above the largest observed total are scaled down to it. Other views are unconstrained.

With total `T` and simulated sample total `t_i`:

```
closed, p >= p0:     X_i <- X_i * T / t_i
otherwise:           X_i <- X_i * min(1, T / t_i)
```

4.6 Relationship to crosstalk

Crosstalk still acts upstream on the latent variables. The two-stage model changes how latent variables are emitted as observed features.

4.7 Relationship to outcome generation

After microbiome features are generated, any outcome mode can be used:

- `LM`,
- `Friedman`,
- `Friedman2`,
- `SharedLatent` with `linear`,
- `SharedLatent` with `friedman`,
- `SharedLatent` with `friedman2`.

5. Spike-In Coefficients

5.1 Goal

Create feature-level beta effects for observed-feature outcome modes.

These effects are used by:

- `LM`,
- `Friedman`,
- `Friedman2`.

They are also stored when using `SharedLatent`, but `SharedLatent` generates the final outcome from shared factors instead of observed-feature betas.

5.2 Number of nonzero effects

For each view:

```
n_de_m ~ Binomial(p_m, de.prob_m).
```

If `de.prob_m > 0` and the draw gives `n_de_m = 0`, it is set to 1, so every view with a positive `de.prob` carries at least one nonzero effect. With `de.prob_m = 0`, all effects in that view are zero.

5.3 Effect magnitudes

For each selected feature:

```
fac_j ~ LogNormal(de.facLoc_m, de.facScale_m).
```

The sign direction is initially sampled using `de.downProb_m`.

The beta value is:

```
beta_j = log2(effect factor).
```

Positive beta means increasing the feature increases the predictor. Negative beta means increasing the feature decreases the predictor.

5.4 Differential-effect alignment through `de.align`

Each feature has a stored:

```
shared_loading_j.
```

This is its loading on the first shared latent direction.

If:

```
de.align = NULL
```

MimeSys uses independent spike-in positions and signs.

If:

```
de.align = a
```

with `0 <= a <= 1`, then:

```
n_aligned = round(a * n_de_m).
```

MimeSys samples those aligned features with probability proportional to:

```
shared_loading_j^2.
```

For aligned features:

```
sign(beta_j) = sign(shared_loading_j).
```

5.5 Interpretation of `de.align`

`de.align` controls whether outcome-relevant observed features point along the shared latent direction.

It is separate from crosstalk:

- crosstalk controls how views are correlated,
- `de.align` controls whether beta effects are aligned with that shared direction.

6. View-Level Signal Balancing

6.1 Goal

Control how much each view contributes to the additive observed-feature predictor.

The argument is:

```
signal.share
```

6.2 View predictors on standardized features

Each simulated feature is standardized within the replicate (mean 0, SD 1; constant features are set to 0), so coefficients are per-SD effects and views measured on very different scales contribute comparably. For each view:

```
eta_m = X~_m beta_m,   X~_m = standardized X_m.
```

6.3 Normalizing requested shares

If:

```
signal.share = c(1, 1, 1)
```

MimeSys normalizes this to:

```
(1/3, 1/3, 1/3).
```

If named values are supplied, they are matched by view name.

6.4 Rescaling beta values

For each view, MimeSys computes:

```
Var(eta_m).
```

If this variance is positive, beta values for that view are multiplied by:

```
sqrt(target_share_m / Var(eta_m)).
```

After this:

```
Var(eta_m) approximately equals target_share_m.
```

6.5 Final additive linear predictor

The additive predictor is:

```
eta_linear = sum_m eta_m.
```

7. Outcome Generation Modes

7.1 `ygen.mode = "LM"`

The outcome signal is:

```
eta = eta_linear = sum_m X_m beta_m.
```

This is an additive observed-feature outcome.

7.2 ygen.mode = "Friedman"

MimeSys selects five observed simulated features.

Candidate features are usually nonzero-beta features:

```
candidate_index = which(beta != 0).
```

If fewer than five are available, it falls back to all features.

The five selected observed features are rank-transformed:

```
U_j = rank(X_j) / (n + 1).
```

Then:

```
eta_friedman =
  10 sin(pi U_1 U_2)
  + 20 (U_3 - 0.5)^2
  + 10 U_4
  + 5 U_5.
```

For `Friedman`:

```
eta = eta_friedman.
```

7.3 ygen.mode = "Friedman2"

MimeSys computes both:

```
eta_linear
```

and:

```
eta_friedman.
```

Both parts are standardized and mixed with `w = nonlinear.weight` (default 0.5):

```
eta = sqrt(1 - w) * std(eta_linear) + sqrt(w) * std(eta_friedman).
```

So this outcome combines additive observed-feature signal and nonlinear observed-feature signal, with `w` roughly the nonlinear share of signal variance (exactly, when the two parts are uncorrelated).

7.4 ygen.mode = "SharedLatent"

The outcome signal is generated from shared latent factors instead of observed features.

During null generation, MimeSys simulates:

```
S_new in R^(n x r).
```

Here:

- `r` is the number of shared latent factors in the template,
- rows are simulated samples,
- columns are shared latent axes.

Are all shared factors used?

By default:

```
shared.latent.n_factors = NULL
```

MimeSys uses all available shared latent factors:

```
K = r.
```

If:

```
shared.latent.n_factors = K
```

MimeSys uses the first `K` shared factors:

```
S_K = [S_1, ..., S_K].
```

If `K > r`, MimeSys stops with an error.

Why multiple shared factors matter

Using only one latent factor can be limiting because cross-omics biology may not be one-dimensional.

Using multiple shared factors allows the outcome to depend on several shared biological axes, for example:

- immune activation,
- metabolic shift,
- proteomic stress response,
- inflammatory pathway activity,
- microbiome-derived metabolite axis.

Shared latent linear outcome

For:

```
shared.latent.fun = "linear"
```

MimeSys standardizes selected shared factors:

```
S_scaled = scale(S_K).
```

Then computes:

```
eta_linear_shared = rowSums(S_scaled) / sqrt(K).
```

The division by `sqrt(K)` prevents variance from increasing just because more factors are included.

The final predictor is:

```
eta = scale(eta_linear_shared).
```

Shared latent Friedman outcome

For:

```
shared.latent.fun = "friedman"
```

MimeSys requires:

```
K >= 5.
```

It takes the first five selected shared factors and rank-transforms them:

```
U_j = rank(S_j) / (n + 1), j = 1, ..., 5.
```

Then:

```
eta_shared_friedman =
  10 sin(pi U_1 U_2)
  + 20 (U_3 - 0.5)^2
  + 10 U_4
  + 5 U_5.
```

The final predictor is:

```
eta = scale(eta_shared_friedman).
```

Shared latent Friedman2 outcome

For:

```
shared.latent.fun = "friedman2"
```

MimeSys computes:

```
eta_linear_shared
```

and:

```
eta_shared_friedman.
```

Then:

```
eta = sqrt(1 - w) * std(eta_linear_shared) + sqrt(w) * std(eta_shared_friedman),
w   = nonlinear.weight (default 0.5).
```

Role of crosstalk in shared-latent outcomes

In `SharedLatent`, the true outcome signal is:

```
Y signal = function(S_new).
```

Crosstalk controls how strongly `S_new` is visible in each observed omics layer.

Low crosstalk means the outcome-driving shared factors may be weakly represented in observed features.

High crosstalk means those shared factors are more strongly reflected in every layer.

Very high crosstalk can make layers redundant, which may not help fusion.

With `crosstalk = 0` the shared factors do not enter any observed feature, so the outcome is unrelated to the features; MimeSys warns in that case.

7.5 `ygen.mode = "Interaction"`

MimeSys selects `K = n.interactions` feature pairs (default: the number of view pairs). Pair `k` cycles over the view pairs `(a, b)` and takes one feature from each view, preferring features with nonzero `beta`. With standardized features `x~`:

```
eta_int = sum_k x~_{a_k} * x~_{b_k},
eta     = sqrt(1 - w) * std(eta_linear) + sqrt(w) * std(eta_int),
w       = interaction.weight (default 0.5).
```

`w = 1` gives a pure cross-view interaction outcome with no main effects. The pairs are stored in `truth$interaction_pairs`.

7.6 Ground truth

`feature_metadata$outcome_feature` marks the features that drive the outcome directly. In `SharedLatent` the outcome depends on the shared latent factors rather than on individual features, so no feature is marked (the latent truth is in `truth$shared`). Components with weight 0 are not marked. `beta` is kept only for features that enter through the linear predictor and is set to zero otherwise (`Friedman`, `SharedLatent`, and `Interaction` with `w = 1`). `friedman_term` and `interaction_term` mark the nonlinear and interaction features.

8. Final Outcome Types

Once `eta` is computed, MimeSys generates the actual outcome.

8.1 Continuous outcome

For:

```
outcome.type = "continuous"
```

MimeSys computes:

```
sigma^2 = Var(eta) / snr.
```

Then:

```
Y_i = eta_i + epsilon_i,
epsilon_i ~ N(0, sigma^2).
```

Large `snr` means lower noise. Small `snr` means higher noise.

8.2 Binary outcome

For:

```
outcome.type = "binary"
```

MimeSys standardizes `eta`, rescales it so that `snr` is the signal-to-noise ratio of the latent logistic model, and solves an intercept `b0` that hits the target prevalence:

```
eta*_i = sqrt(snr * pi^2 / 3) * (eta_i - mean(eta)) / sd(eta),
b0 solves mean(logistic(b0 + eta*_i)) = binary.prevalence,
Y_i ~ Bernoulli(logistic(b0 + eta*_i)).
```

8.3 Survival outcome

For:

```
outcome.type = "survival"
```

MimeSys standardizes `eta` and rescales it so that `snr` is the signal-to-noise ratio on the log event-time scale (the extreme-value error of an exponential model has variance `pi^2 / 6`):

```
eta*_i = sqrt(snr * pi^2 / 6) * (eta_i - mean(eta)) / sd(eta),
h_i = surv.hscale exp(eta*_i),
T_i ~ Exponential(rate = h_i).
```

Censoring is:

```
C_i ~ Uniform(cens.lower, cens.upper)          if cens.rate is NULL,
C_i = U_i c,  U_i ~ Uniform(0, 1)               otherwise,
```

where `c` is solved so that the proportion with `C_i < T_i` equals `cens.rate` (`cens.rate = 0` means no censoring).

Observed output is:

```
time_i = min(T_i, C_i)
status_i = 1 if T_i <= C_i, 0 otherwise.
```

9. Train/Test Split

MimeSys samples:

```
round(nsample * p.train)
```

samples for training.

It then subsets:

- feature table,
- sample metadata,
- factor scores,
- shared latent factors,
- outcome metadata.

The outcome is generated once before splitting.

10. Crosstalk Calibration

`calibrate_crosstalk()` is used when the user wants to choose crosstalk by reconstruction of empirical cross-view correlation.

The process is:

1. Choose a grid of candidate crosstalk values (default `0, 0.25, ..., 1`); values above `crosstalk_max()` are dropped with a warning.
2. For each value, simulate `nrep` (default 5) independent null data sets with `nsample` samples (default the template size). Each view needs at least two features.
3. Align simulated and observed features by template feature name (offspring are not compared) and compute, for every pair of views `(a, b)`, the block distance

   ```
   d_ab = sqrt( mean over finite entries of (R_sim_ab - R_obs_ab)^2 ),
   ```

   the rescaled Frobenius norm of the difference of the feature-correlation blocks (diagonal excluded within a view). Observed blocks are computed once and cached.
4. For each replicate, average `d_ab` over all cross-view blocks; average over replicates to get the mean, SD and SE for each grid value.
5. Select the value with the smallest mean. Values within one SE of it are `near_optimal`, and `selection_frequency` gives how often each value is best in individual replicates. A warning is given when the best value is at the top of a grid that stops well short of `crosstalk_max()`.

Calibration always compares with the matrices stored in the template (fitted scale). It needs at least two views; a template without shared factors gives a warning. The MAE wrappers calibrate by default (`crosstalk = "calibrate"`), skip calibration for a single view, no shared factors or fewer than two features per view, and run it with seed + 1 so its replicates do not share random numbers with the simulation that follows.

This selects crosstalk for feature-structure reconstruction, not for predictive performance. With small templates the curve is often shallow; then report results over a range of crosstalk values.

10.1 Template validation

`validate_template()` calibrates crosstalk, simulates one null data set at the selected value (seed + 1), keeps the `max_features_per_view` (default 100) most variable observed features and the same simulated features, and reports:

- `reconstruction_error`: the block distances above;
- `correlation_structure`: per block, the Pearson and Spearman correlation between observed and simulated feature-feature correlations, MAE and RMSE (within-view blocks currently include the self-pairs, which slightly inflates them);
- `marginal_diagnostics`: per view, the mean and median Kolmogorov-Smirnov distance, the standardized and raw quantile RMSE over 19 probabilities.

10.2 Reproducibility

Every function that uses random numbers has `seed = 1234` by default and restores the caller's random-number state on exit (`withr::local_seed()`); `seed = NULL` uses the current stream. Internal loops call the unseeded engines, so replicates are independent draws.

11. Summary of Controls

`rank`:

- within-view factor rank,
- if `NULL`, chosen by variance explained,
- not `2 log(p)` by default.

`shared.rank`:

- number of shared cross-view latent factors,
- if `NULL`, chosen by permutation-based parallel analysis,
- if numeric, forced to that value.

`crosstalk`:

- strength of cross-view feature dependence,
- `0` removes shared latent dependence,
- `1` uses fitted template shared structure.

`de.align`:

- aligns beta effects with shared latent loading direction.

`signal.share`:

- controls how much each view contributes to additive observed-feature signal.

`friedman.share`:

- controls which views supply the five observed features used in Friedman/Friedman2.

`shared.latent.n_factors`:

- controls how many shared latent factors are used for SharedLatent outcomes,
- `NULL` means use all available shared factors.

`shared.latent.fun`:

- `"linear"`: linear shared-latent score,
- `"friedman"`: nonlinear Friedman function on shared factors,
- `"friedman2"`: linear plus nonlinear shared-latent signal.

12. Mediation Simulation

`gen_mediation()` and `simulate_mediation()` add an exposure, mediators and an outcome to null multi-view data.

12.1 Exposure

`T` is binary (`P(T = 1) = pi`) or standard normal. Its standardized version is `T* = (T - mu_T) / sd_T`.

12.2 Exposure to mediators

For mediator `j` with latent normal score `Z_j` from the template:

```
Z_j <- a_j T* + sqrt(1 - a_j^2) Z_j.
```

For a continuous exposure this is again standard normal. For a binary exposure it is a two-component normal mixture with CDF

```
F(x) = (1 - pi) Phi((x - a_j t0*) / s_j) + pi Phi((x - a_j t1*) / s_j),
s_j = sqrt(1 - a_j^2),
```

and `Z_j <- Phi^-1(F(Z_j))` restores an exact standard normal. The latent score is then mapped through the feature's empirical quantile function, so the mediator keeps its template marginal while the exposure shifts it.

12.3 Outcome

With mediators standardized by their template mean and SD (`M~_j`):

```
continuous: Y = DE T + sum_j b_j M~_j + sum_k g_k M~_k1 M~_k2 + sigma e,
binary:     logit P(Y = 1) = b0 + DE T + sum_j b_j M~_j + sum_k g_k M~_k1 M~_k2.
```

12.4 Path sizes and calibration

Initial paths are `|a_j| = |b_j| = s = sqrt(TIE sd_T / K)` (K mediators), with signs from `dir.a` and `dir.b`; interaction coefficients have size `mean(|b|)`. Paired counterfactual Monte Carlo (`n.mc` draws under `T = 1` and `T = 0` with the same random stream) gives the exposure effect on every standardized feature, `Delta_j = E[M~_j(1)] - E[M~_j(0)]`. The b-paths and interaction coefficients are then scaled by one factor so that

```
NIE = sum_j b_j Delta_j + sum_k g_k Delta(M~_k1 M~_k2) = TIE dir.a dir.b,
```

or, with a random direction, so that the summed absolute pathway effects equal `TIE`. In `null_a` the same b-paths are used with `a = 0`; in `null_b` the a-paths are kept and `b = 0`.

12.5 Truth

- `NIE`, `NDE = DE`, `TE`, the indirect effect per view and through interactions, and per feature `a_path`, `a_effect`, `a_slope`, `b_path`, `indirect` and `exposure_effect`.
- Continuous exposure: `NIE_linear = sum_j b_j a_slope_j`, with `a_slope_j` the population linear regression slope of `M~_j` on `T`, which is what product-of-coefficients estimators target.
- Binary outcome: `NIE`, `NDE`, `TE` are conditional (linear-predictor) log-odds effects. The natural effects, from counterfactual risks `p(t, t') = E[P(Y = 1 | T = t, M(t'))]`, are reported as risk differences, odds ratios and log odds ratios in `truth$mediation$binary`.
- In closed compositional views, renormalization transmits the exposure to non-mediators; their true effect is in `exposure_effect`.
- All truth refers to the fitted (transformed) scale.

12.6 Sequential mediation

With `stages`, the mediator views are ordered, e.g. `T -> M1 -> M2 -> Y` for stages `list("Microbiome", "Metabolome")`. Write `W_j` for the latent score of mediator `j` before the binary-exposure correction. Stage-1 mediators are as in 12.2, `W_j = a_j T* + sqrt(1 - a_j^2) Z_j`. A stage-`s` mediator (`s > 1`) with parents `k` in stage `s - 1` is

```
W_j = a_j T* + sum_k d_jk W_k + c_j Z_j.
```

Every `W` is linear in `T*` and the template latent scores, `W_j = mu_j T* + L_j Z`, with `Cov(Z) = Sigma` from the fitted loadings and crosstalk (`Cov(Z_i, Z_k) = Lambda_i Cov(f_m, f_m') Lambda_k'`). Given `T`, `W_j` is normal with mean `mu_j T*`, where `mu_j = a_j + sum_k d_jk mu_k`. `c_j > 0` solves `mu_j^2 + Var(L_j Z) = 1`, so each `W_j` has unit variance, and the binary-exposure correction of 12.2 is applied with `mu_j` in place of `a_j`. Every mediator therefore keeps its template marginal.

Links. A sparse signed pattern (rows: children, columns: parents) is drawn by `stage.links$type`: `"random"` (each pair with probability `density`, every child with at least one parent and every parent with at least one child), `"hub"` (a `density` fraction of the parents, at least one, are hubs; every child gets one hub parent, and every hub gets a child when there are at least as many children as hubs), `"convergent"` (each child gets `max(2, density * n_parents)` parents) or `"explicit"`. Each child's row is scaled so that the parents' term has variance `strength`, `d_j' R d_j = strength`, with `R` the parents' latent covariance (close to the share of the child's latent variance explained by its parents when that term is nearly uncorrelated with the exposure and the child's own score). The links are fixed in the spiked scenario and reused in the null scenarios.

Natural path-specific effects. A path `(r, s)` starts at stage `r` and reaches `Y` through the stage-`s` mediators. For each observed stage `s`, the chain up to `s` is built from its own copy of the exposure, so each path can be switched on separately (cross-world counterfactuals with shared random draws). Paths are switched from the unexposed to the exposed level one at a time, later-starting paths first; for two stages:

```
T -> M2 -> Y:       E[Y(1, M1(0), M2(1, M1(0)))] - E[Y(1, M1(0), M2(0, M1(0)))]
T -> M1 -> M2 -> Y: E[Y(1, M1(0), M2(1, M1(1)))] - E[Y(1, M1(0), M2(1, M1(0)))]
T -> M1 -> Y:       E[Y(1, M1(1), M2(1, M1(1)))] - E[Y(1, M1(0), M2(1, M1(1)))]
```

These telescope to the NIE. Each is linear in the b-paths given the a-paths and links, so `stage.share` is met by scaling the b-paths of each stage separately (paths ending at stage `s` scale with the stage-`s` b-paths).

Truth. `path_effects`, `NIE_by_stage`, per-feature `stage`, `n_parents` and `n_children`, and the link list with each latent coefficient and an approximate per-link chain effect `(mu_parent / sd_T) d_jk b_child` (a latent-scale linear approximation; the exact totals are in `path_effects`). For a binary outcome the path-specific risk differences are also reported.

Estimation on the data scale. The links act on the latent scores. For roughly Gaussian features the data are close to linear in them and product-of-coefficients estimates recover each path. For strongly skewed or sparse features (e.g. microbiome relative abundances) the observed parent is a non-linear function of its latent score, so a linear model on the observed values captures only part of the chain, and the remainder appears as a direct `T -> M2` effect. This is a property of the data, not of the truth, and is worth keeping in mind when benchmarking linear sequential methods (transforming such features, e.g. ranks or log ratios, helps).

Crosstalk as mediator-mediator confounding. The template latent scores of stage-1 and stage-2 mediators are correlated through the shared factors, a common cause of both that is not on any path from `T`. The path-specific truth holds it fixed correctly, but a regression of `M2` on `T` and `M1` is confounded by it. On the Stelzer template (n = 30,000, TIE = 1) the chain `T -> M1 -> M2 -> Y` was 0.23 (truth) but estimated as 0.14 with crosstalk 1 and 0.19 with crosstalk 0, the difference moving to the direct `T -> M2 -> Y` path. Varying `crosstalk`, and the `strength = 0` scenario, separate this from real links.

13. Recommended Controlled Shared-Latent Setup

For controlled nonlinear shared-latent validation:

```
template <- fit_template(
  views,
  rank = 10,
  shared.rank = 5
)

sim <- gen_simmba(
  nsample = 200,
  template = template,
  p = c(100, 100, 100),
  crosstalk = 0.75,
  ygen.mode = "SharedLatent",
  shared.latent.fun = "friedman2",
  shared.latent.n_factors = 5,
  snr = 0.5
)
```

This makes the shared latent structure explicit:

- exactly five shared factors are fitted,
- exactly five shared factors are used for the outcome,
- the outcome contains both linear and nonlinear shared-latent signal,
- crosstalk controls how strongly that outcome-driving latent structure appears in each observed omics layer.